AT8236 电机驱动

用户手册

版本说明：

版本	日期	内容说明
1.0	2022/11/10	第一次发布
1.1	2023/04/03	内容勘误
1.2	2023/04/17	内容勘误
1.3	2023/08/29	内容勘误
1.4	2025/06/11	内容勘误

序言

我们将通过这篇教程与大家一起学习直流电机的原理和控制、减速器的作用，并介绍一款直流电机驱动芯片 AT8236。

目录

1. 直流电机入门基础知识 4
1.1 直流电机原理 4
1.2 减速器 5
1.3 电机实物接线图解 5
2. AT8236 模块介绍 7
2.1 D107A 模块说明 7
2.2 D157B 模块说明 7
这个模块集成了以下的功能： 7
3. 原理图说明 9
3.1 AT8236 芯片原理图介绍与控制说明 9
3.2 D107A 原理图介绍 11
3.3 D157B 原理图介绍 12
4. 接线说明 13
4.1 D107A 模块接线说明 13
4.2 D157B 模块接线说明 14
5. 代码例程 16
5.1 D107A 与 STM32F103C8T6 的例程代码 16
5.2 D157B 与 STM32F103C8T6 的例程代码 17

1. 直流电机入门基础知识

1.1 直流电机原理

下面是分析直流电机的物理模型图。其中，固定部分有磁铁，这里称作主磁极；固定部分还有电刷。转动部分有环形铁心和绕在环形铁心上的绕组。(其中 2 个小圆圈是为了方便表示该位置上的导体电势或电流的方向而设置的)

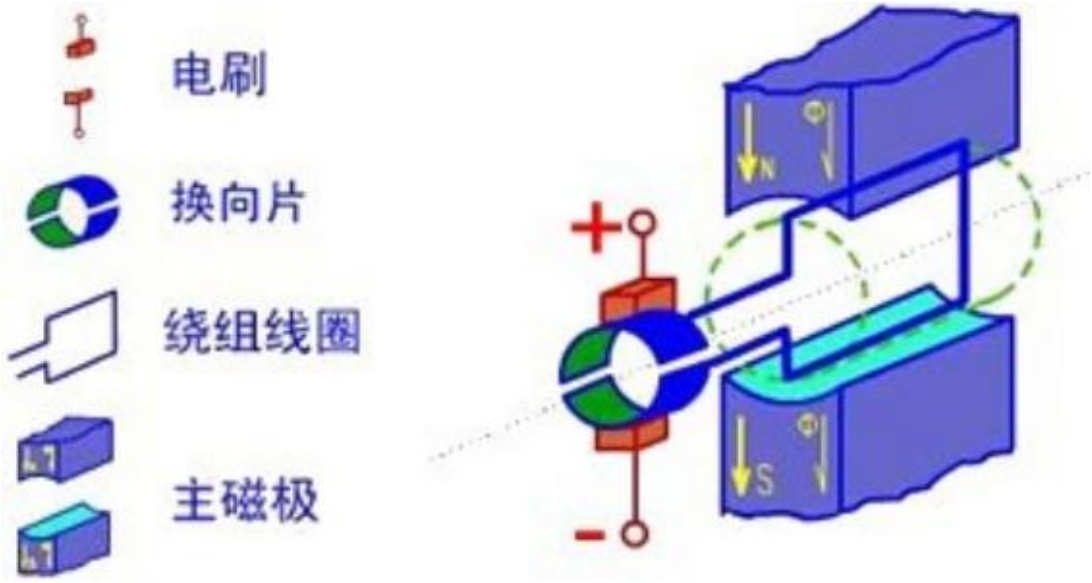

图 1-1-1 直流电机的物理模型图

它的固定部分（定子）上，装设了一对直流励磁的静止的主磁极 N 和 S，在旋转部分（转子）上装设电枢铁心。在电枢铁心上放置了两根导体连成的电枢线圈，线圈的首端和末端分别连到两个圆弧形的铜片上，此铜片称为换向片。换向片之间互相绝缘，由换向片构成的整体称为换向器。换向器固定在转轴上，换向片与转轴之间亦互相绝缘。在换向片上放置着一对固定不动的电刷B1 和 B2，当电枢旋转时，电枢线圈通过换向片和电刷与外电路接通。

在电刷上施加直流电压 U,电枢线圈中的电流流向为：N 极下的有效边中的电流总是一个方向，而 S 极下的有效边中的电流总是另一个方向。这样两个有效边所受的洛伦兹力的方向一致（可以根据左手法则判定），电枢开始转动。

具体就是上图中的+和-分别接到电池的正极和负极，电机即可转动；如果是把上图中的+和-分别接到电池的负极和正极，则电机会反方向转动。电机的转速可以理解为和外接的电压是正相关的（实际是由电枢电流决定）。

总结，如果我们可以调节施加在电机上面的直流电压大小，即可实现直流电

机调速，变施加电机上面直流电压的极性，即可实现电机换向。

1.2 减速器

一般直流电机的转速都是一分钟几千上万转的，所以一般需要安装减速器。减速器是一种相对精密的机械零件，使用它的目的是降低转速，增加转矩。减速后的直流电机力矩增大、可控性更强。按照传动级数不同可分为单级和多级减速器；按照传动类型可分为齿轮减速器、蜗杆减速器和行星齿轮减速器。

图 1-2-1 减速器类型

齿轮减速箱体积较小，传递扭矩大，但是有一定的回程间隙。蜗轮蜗杆减机的主要特点是具有反向自锁功能，可以有较大的减速比，但是一般体积较大，传动效率不高，精度不高。行星减速机其优点是结构比较紧凑，回程间隙小、精度较高，使用寿命很长，额定输出扭矩可以做的很大，但价格略贵。

以下是一款搭配多级齿轮减速箱的电机。

图 1-2-2 多级齿轮减速箱的电机

1.3 电机实物接线图解

具体到我们的电机，我们可以看看电机后面的图解。

图 1-3-1 电机接线的详细图解

上面介绍了一大堆说直流电机只引出两个线，怎么这个电机有 6 个线，而且还有两个大焊点呢？其实，根据上面的图解也知道，那两个焊点分别和黄线和棕线是连接在一起的。也就是说只有 6 个线，而 6P 排线中，中间的四根线（红绿白黑）是编码器的线，只是用于测速，和直流电机本身没有联系。我们在实现开环控制的时候无需使用。

综上所述，我们只需控制施加在黄线和棕色线两端的直流电压大小和极性即可实现调试和换向。

2. AT8236 模块介绍

(1)　AT8236 是一款直流有刷的电机驱动器件，能够以高达 6A 的峰值电流双向控制电机。利用电流衰减模式，可通过对输入信号进行脉宽调制（PWM）来控制电机的转速，同时具备低功耗休眠功能。

(2)　AT8236 集成同步整流功能，可显著降低系统的功耗要求。内部保护功能包括过流保护，短路保护，欠压锁定和过温保护。

(3)　AT8236 的特点：

- 单通道 H 桥电机驱动器
- 宽电压供电，5.5V-36V
- 低 $R_{DS(ON)}$ 电阻，200mΩ（HS+LS）
- 6A 峰值驱动输出，4A 连续驱动输出
- PWM 控制接口
- 支持低功耗休眠功能

2.1 D107A 模块说明

这个模块是由两个 AT8236 组成，背面有一个电容，主要起稳压作用。

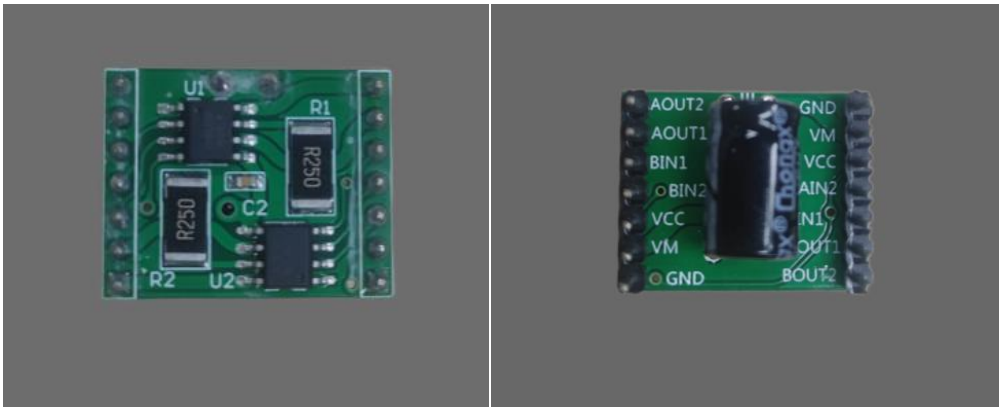

图 2-1-1 D107A 模块详细图

2.2 D157B 模块说明

这个模块集成了以下的功能：

（1）新增了 5V 稳压电路，支持 5V 和 3.3V 的输出。5V 和 3.3V 各有 3 个这样的引脚可以对外供电。

（2）增加电压测量电路，通过串联一个 10kΩ和一个 1kΩ的电阻，对输入电源进行 1/11 的分压后，可以通过 ADC 采集并计算得到电源的电压进行监控。

（3）引出电机标准的 6PIN 接口，可以通过用排线连接，AB 相编码器的信号接到单独的引脚输出。

（4）新增了电源输出电路，电源输入接口做了并联输出，可以多个模块级联使用。

（5）含有电源开关，可以进行开启。

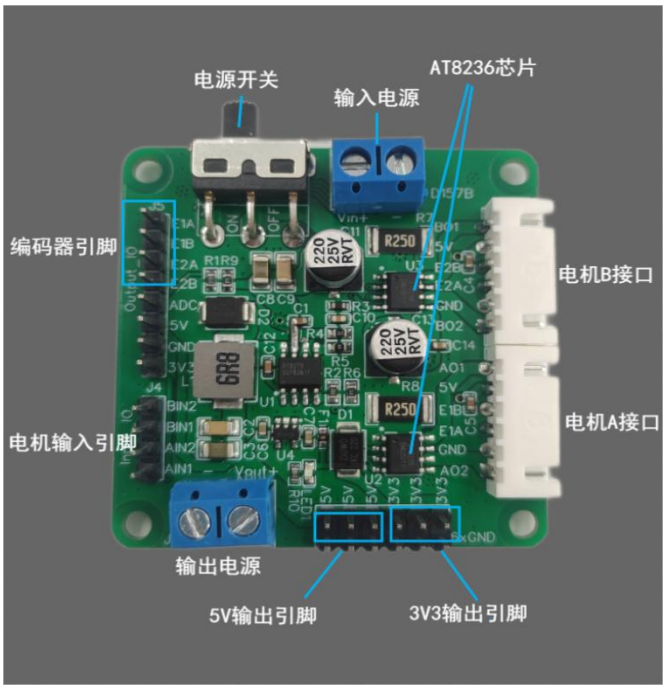

图 2-2-1 AT8236 带稳压模块详细图

3. 原理图说明

3.1 AT8236 芯片原理图介绍与控制说明

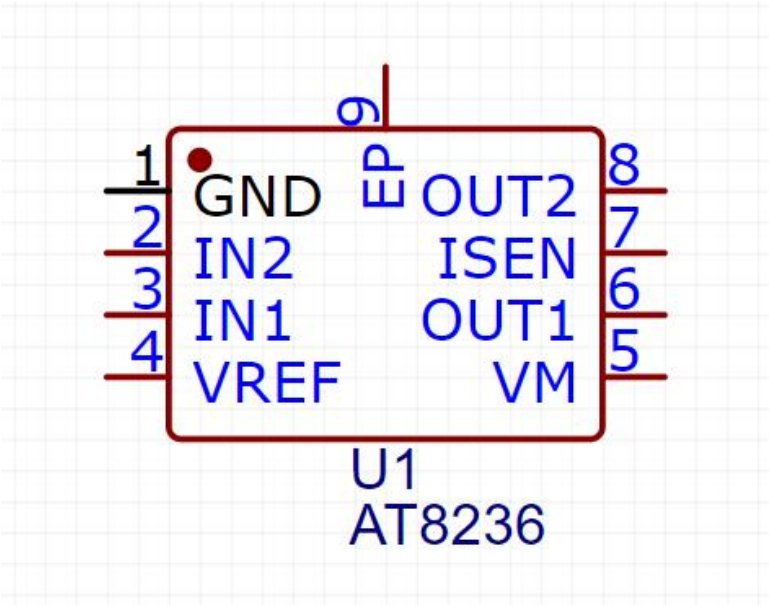

图 3-1-1　AT8236 芯片原理图

表 3-1-1 AT8236 引脚表

引脚	功能
VM	电机驱动电压输入（5.5V-17V）
VREF	逻辑电平输入端（0.5V-4V）
GND	接地
IN1	控制信号输入 1
IN2	控制信号输入 2
OUT1	电机输出端 1
OUT2	电机输出端 2
ISEN	H 桥检流输入/地
EP	芯片底部的焊盘，散热用

在使用的过程中，VM 需要比较大的电压，如 12V，而且 VREF 还需要接 3.3V。VM 是用来给电机供电的，VREF 是给芯片供电的，切记勿要接错，否则将会烧坏芯片。在控制电机时，OUT1 和 OUT2 连接电机的两个引脚，这两个引脚可以控制电机的转速（PWM 输入控制），也可以控制电机的转向。

输入管脚 IN1、IN2 控制 H 桥的输出状态。下表是输入输出间的逻辑关系。

表 3-1-2 AT8236 输入输出的逻辑关系

IN1	IN2	OUT1	OUT2	功能
0	0	Z	Z	滑行，休眠
1	0	H	L	正向
0	1	L	H	反向
1	1	L	L	刹车

当时使用 PWM 控制来实现调速功能时，H 桥可以操作在两种不同的状态，快衰竭或者慢衰竭（单极性或双极性）。在快衰竭（双极性）的模式，H 桥是被禁止的模式的，续流电流流经二极管；在慢衰竭（单极性）模式下，输出 H 桥的两个下管是打开的。

表 3-1-3 AT8236PWM 调速

IN1	IN2	功能
PWM	0	正转 PWM，快衰竭
1	PWM	正转 PWM，慢衰竭
0	PWM	反转 PWM，快衰竭
PWM	1	反转 PWM，慢衰竭

这个快衰竭、慢衰竭模式（单极性、双极性）。

慢衰竭（单极性）：

优点：控制电路简单。

缺点：不能刹车，不能能耗制动，在负载超过设定速度时不能提供向力矩。调速静差大，调速性能很差，稳定性也不好。

当 PWM 为高电平时：MOS 管 Q1 和 Q4 导通，MOS 管 Q2 和 Q3 截止，电流从电源正极经过 Q1，从坐到右流过电机，然后经过 Q4 回到电源负极。

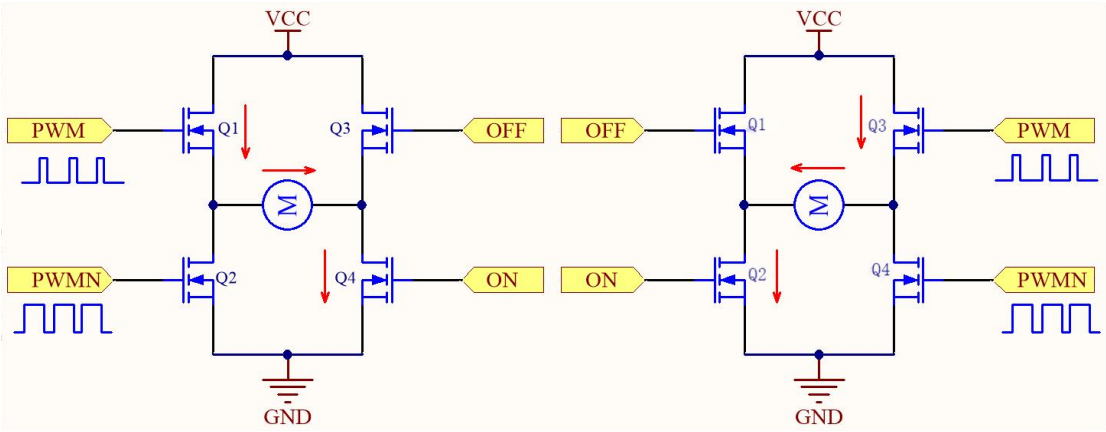

图 3-1-2 单极性模式

快衰竭（双极性）：

优点：能正反转运行，启动快，调速精度高，动态性能好，调速静差小，调速范围大，能加速，减速，刹车，倒转，能在负载超过设定速度时，提供反向为矩，能克服电机轴承的静态摩擦力，产生非常低的转速。

缺点：控制电路复杂。在工作期间，4 个 MOS 管都处于工作状态，功率损耗大，电机容易发烫。

PWM1 和 PWM1N、PWM2 和 PWM2N 是互补通道，PWM1 和 PWM2 周期相同，占空比相同，极性相反，使得对角的两个 MOS 管同时导通和截止。

双极性需要采用高级定时器来完成。

3.2 D107A 原理图介绍

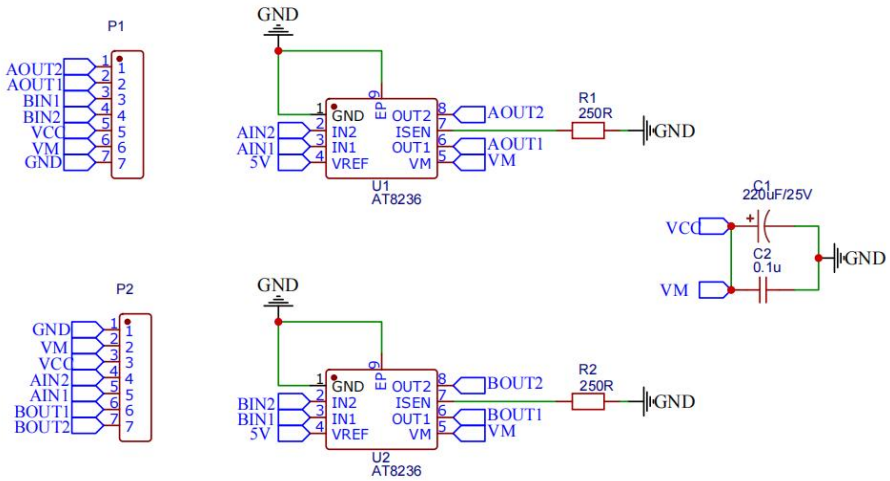

图 3.2-1 D107A 原理图

3.3 D157B 原理图介绍

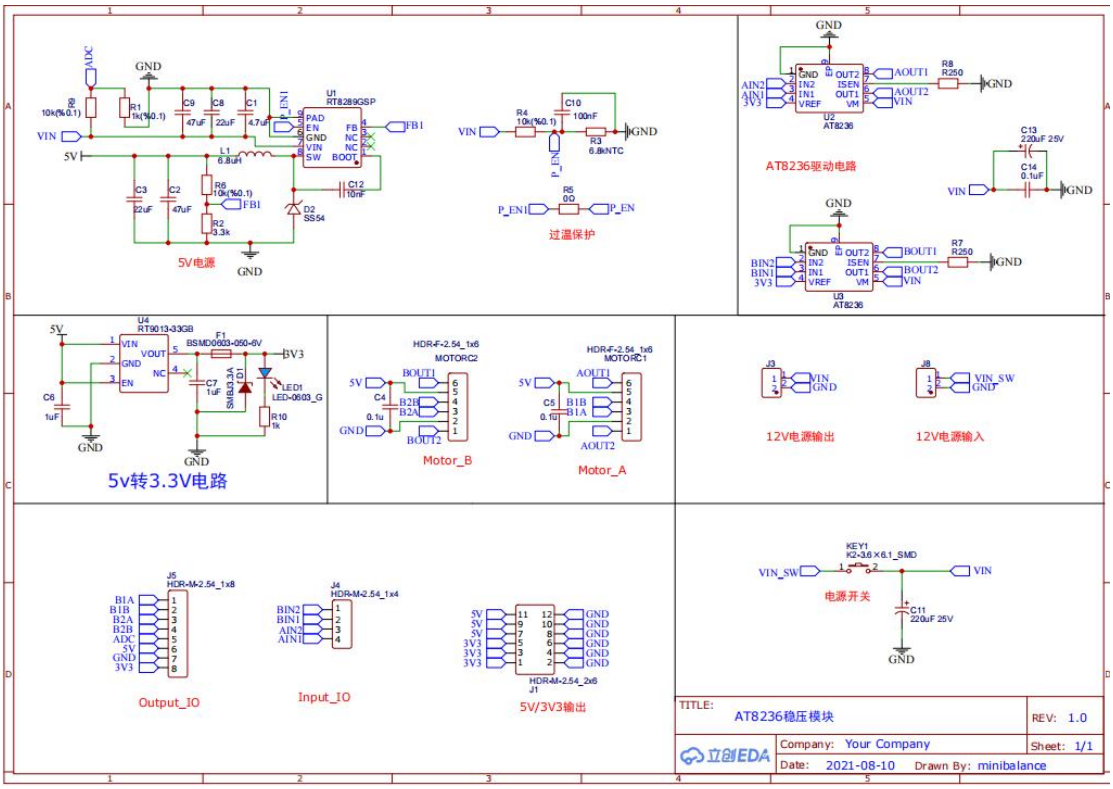

图 3-2-1 D157B 原理图

D157B 模块多了稳压电路、电机接口、12V 电源输入输出、5V 电源和 3.3V 电源的并联输出和电源开关。这就可以直接和一个单片机的最小模块直接使用，可以采集 ADC，对模块的电源进行监控；有了电机接口，可以直接插上电机线，对电机的编码器数据读取，还只需要接一个外部电源，再由 D157B 的 5V 电源输出为单片机供电，比较简便。

4. 接线说明

提供的接线说明都是基于本公司产品的例程，我们提供了 STM32F107C8T6 例程一份。别的产品需要自己摸索一下，但也只需改动引脚，只要是和本例程功能相同的引脚即可。

4.1 D107A 模块接线说明

D107A 与 STM32F103C8T6 接线引脚如下图所示：

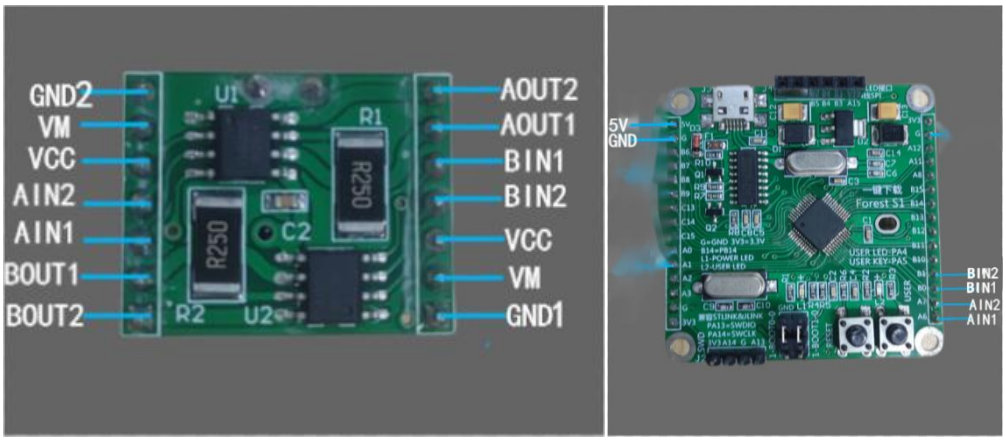

图 4.1-1 各模块引脚图

表 4-1-1 D107A 模块与 STM32F1 接线表

Stm32F1	D107A 模块	电机	
	VM		5V~15V 外接电源+
3.3V	VCC		外接 3.3V 输入电源
GND	GND2		
	AOUT1	电机电源+	
	AOUT2	电机电源-	
	BOUT1	电机电源+	
	BOUT2	电机电源-	
PB0	BIN1		
PB1	BIN2		
PA6	AIN1		
PA7	AIN2		
	GND1		外接地

这里的 VM、VCC 只需要各接上一个即可。

4.2 D157B 模块接线说明

D157B 带稳压模块与 STM32F103C8T6 接线引脚如下图所示：

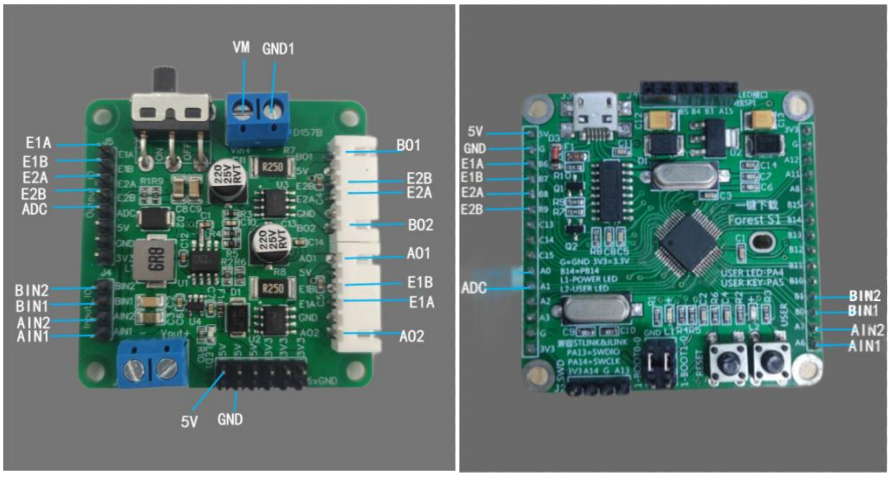

图 4-2-1 各模块引脚图

表 4-2-1 D157B 模块与 STM32F1 接线表

电机(用 6pin 接线连接)	D157B	Stm32F103C8T6	
	ADC	PA2	
	AIN1	PA6	
	AIN2	PA7	
	BIN1	PB0	
	BIN2	PB1	
	VM		5V~15V 外接电源
	GND1		外接地
	5V	5V	
	GND	GND	
编码器 B 相	E2B	B6	
编码器 A 相	E2A	B7	
编码器 B 相	E1B	A1	
编码器 A 相	E1A	A0	
电机线-	A01		
电机线+	A02		
电机线+	B01		
电机线-	B02		

图 4-2-2 电机和 D157B 接线示意图

5. 代码例程

5.1 D107A 与 STM32F103C8T6 的例程代码

```
void PWM_Int(u16 arr,u16 psc)
{
    GPIO_InitTypeDef GPIO_InitStructure;                //定义结构体GPIO_InitStructure
    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;      //定义结构体TIM_TimeBaseStructure
    TIM_OCInitTypeDef TIM_OCInitStructure;              //定义结构体TIM_OCInitStructure

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB|RCC_APB2Periph_GPIOA,ENABLE);//使能PB端口时钟
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3,ENABLE);//使能定时器3

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;         //复用模式输出
    GPIO_InitStructure.GPIO_Pin  = GPIO_Pin_0|GPIO_Pin_1; //PB0 、PB1
    GPIO_InitStructure.GPIO_Speed= GPIO_Speed_2MHz;         //IO口速度
    GPIO_Init(GPIOB,&GPIO_InitStructure);           //GPIO初始化

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Pin  = GPIO_Pin_6|GPIO_Pin_7;
    GPIO_InitStructure.GPIO_Speed= GPIO_Speed_2MHz;
    GPIO_Init(GPIOA,&GPIO_InitStructure);

    TIM_TimeBaseStructure.TIM_Period = arr;                 //设置下一个更新活动的自动重装载寄存器的值
    TIM_TimeBaseStructure.TIM_Prescaler = psc;              //预分配值
    TIM_TimeBaseStructure.TIM_ClockDivision = 0;            //时钟分割
    TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up; //向上计数
    TIM_TimeBaseInit(TIM3,&TIM_TimeBaseStructure);

    TIM_OCInitStructure.TIM_OCMode= TIM_OCMode_PWM1;            //PWM脉冲宽度调制1
    TIM_OCInitStructure.TIM_Pulse = 0;                          //设置待装入捕获比较寄存器的脉冲值
    TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;   //设置TIM输出极性为高
    TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;//比较输出使能
    TIM_OC1Init(TIM3,&TIM_OCInitStructure);
    TIM_OC2Init(TIM3,&TIM_OCInitStructure);
    TIM_OC3Init(TIM3,&TIM_OCInitStructure);
    TIM_OC4Init(TIM3,&TIM_OCInitStructure);

    TIM_CtrlPWMOutputs(TIM3,ENABLE);//主输出使能

    TIM_OC1PreloadConfig(TIM3,TIM_OCPreload_Enable);
    TIM_OC2PreloadConfig(TIM3,TIM_OCPreload_Enable);
    TIM_OC3PreloadConfig(TIM3,TIM_OCPreload_Enable);
    TIM_OC4PreloadConfig(TIM3,TIM_OCPreload_Enable);//使能预装载寄存器

    TIM_ARRPreloadConfig(TIM3,ENABLE);              //使能自动装载允许位
    TIM_Cmd(TIM3,ENABLE);//启动定时器3
}
```

图 5-1-1 PWM 的初始化

这个模块只需要初始化定时器 3 的 4 个通道为 PWM 输出即可。

```
void moto(int mode)
{
    if(mode==1)     //反转
    {
      PWMA_IN1=7200;PWMA_IN2=5000;
      PWMB_IN1=7200;PWMB_IN2=5000;
    }
     if(mode==0)    //正传
    {
      PWMA_IN2=7200;PWMA_IN1=5000;
      PWMB_IN2=7200;PWMB_IN1=5000;
     }

}
```

图 5-1-2 电机的控制

这里是对应了表 3-1-3 中的慢衰竭模式，一个为高电平，一个为 PWM 输入。

5.2 D157B 与 STM32F103C8T6 的例程代码

```
void PWM_Int(u16 arr,u16 psc)
{
    GPIO_InitTypeDef GPIO_InitStructure;                    //定义结构体GPIO_InitStructure
    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;          //定义结构体TIM_TimeBaseStructure
    TIM_OCInitTypeDef TIM_OCInitStructure;                  //定义结构体TIM_OCInitStructure

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB|RCC_APB2Periph_GPIOA,ENABLE);//使能PB端口时钟
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3,ENABLE);//使能定时器3

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;          //复用模式输出
    GPIO_InitStructure.GPIO_Pin  = GPIO_Pin_0|GPIO_Pin_1; //PB0 、PB1
    GPIO_InitStructure.GPIO_Speed= GPIO_Speed_2MHz;          //IO口速度
    GPIO_Init(GPIOB,&GPIO_InitStructure);            //GPIO初始化

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Pin  = GPIO_Pin_6|GPIO_Pin_7;
    GPIO_InitStructure.GPIO_Speed= GPIO_Speed_2MHz;
    GPIO_Init(GPIOA,&GPIO_InitStructure);

    TIM_TimeBaseStructure.TIM_Period = arr;                 //设置下一个更新活动的自动重装载寄存器的值
    TIM_TimeBaseStructure.TIM_Prescaler = psc;              //预分配值
    TIM_TimeBaseStructure.TIM_ClockDivision = 0;            //时钟分割
    TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up; //向上计数
    TIM_TimeBaseInit(TIM3,&TIM_TimeBaseStructure);

    TIM_OCInitStructure.TIM_OCMode= TIM_OCMode_PWM1;              //PWM脉冲宽度调制1
    TIM_OCInitStructure.TIM_Pulse = 0;                            //设置待装入捕获比较寄存器的脉冲值
    TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;     //设置TIM输出极性为高
    TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;//比较输出使能
    TIM_OC1Init(TIM3,&TIM_OCInitStructure);
    TIM_OC2Init(TIM3,&TIM_OCInitStructure);
    TIM_OC3Init(TIM3,&TIM_OCInitStructure);
    TIM_OC4Init(TIM3,&TIM_OCInitStructure);

    TIM_CtrlPWMOutputs(TIM3,ENABLE);//主输出使能

    TIM_OC1PreloadConfig(TIM3,TIM_OCPreload_Enable);
    TIM_OC2PreloadConfig(TIM3,TIM_OCPreload_Enable);
    TIM_OC3PreloadConfig(TIM3,TIM_OCPreload_Enable);
    TIM_OC4PreloadConfig(TIM3,TIM_OCPreload_Enable);//使能预装载寄存器

    TIM_ARRPreloadConfig(TIM3,ENABLE);              //使能自动装载允许位
    TIM_Cmd(TIM3,ENABLE);//启动定时器3
```

图 5-2-1 D157B 电机 PWM 的初始化

D157B 的 PWM 使用的是定时器 3，因为 AT8236 是由两个 IO 口来控制电机的转速和方向的，就像表 3-1-3 中的慢衰竭模式。所以这里只需要讲两个电机的 4 个 IO 口进行 PWM 初始化即可。

```
void Encoder_Init_Tim2(void)
{
    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
  TIM_ICInitTypeDef TIM_ICInitStructure;
  GPIO_InitTypeDef GPIO_InitStructure;
  RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);//使能定时器4的时钟
  RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);//使能PB端口时钟

  GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0|GPIO_Pin_1;  //端口配置
  GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING; //浮空输入
  GPIO_Init(GPIOA, &GPIO_InitStructure);                          //根据设定参数初始化GPIOB

  TIM_TimeBaseStructInit(&TIM_TimeBaseStructure);
  TIM_TimeBaseStructure.TIM_Prescaler = 0x0; // 预分频器
  TIM_TimeBaseStructure.TIM_Period = 65535; //设定计数器自动重装值
  TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1;//选择时钟分频：不分频
  TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;////TIM向上计数
  TIM_TimeBaseInit(TIM2, &TIM_TimeBaseStructure);
  TIM_EncoderInterfaceConfig(TIM2, TIM_EncoderMode_TI12, TIM_ICPolarity_Rising, TIM_ICPolarity_Rising);//使用编码器模式3
  TIM_ICStructInit(&TIM_ICInitStructure);
  TIM_ICInitStructure.TIM_ICFilter = 10;    //滤波10
  TIM_ICInit(TIM2, &TIM_ICInitStructure);
  TIM_ClearFlag(TIM2, TIM_FLAG_Update);//清除TIM的更新标志位
  TIM_ITConfig(TIM2, TIM_IT_Update, ENABLE);
  //Reset counter
  TIM_SetCounter(TIM2,0);
  TIM_Cmd(TIM2, ENABLE);
}
```

图 5-2-2 编码器 TIM2 的初始化

```c
void Encoder_Init_Tim4(void)
{
    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
  TIM_ICInitTypeDef TIM_ICInitStructure;
  GPIO_InitTypeDef GPIO_InitStructure;
  RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM4, ENABLE);//使能定时器4的时钟
  RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);//使能PB端口时钟

  GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6|GPIO_Pin_7;  //端口配置
  GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING; //浮空输入
  GPIO_Init(GPIOB, &GPIO_InitStructure);                          //根据设定参数初始化GPIOB

  TIM_TimeBaseStructInit(&TIM_TimeBaseStructure);
  TIM_TimeBaseStructure.TIM_Prescaler = 0x0; // 预分频器
  TIM_TimeBaseStructure.TIM_Period = 65535; //设定计数器自动重装值
  TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1;//选择时钟分频，不分频
  TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;////TIM向上计数
  TIM_TimeBaseInit(TIM4, &TIM_TimeBaseStructure);
  TIM_EncoderInterfaceConfig(TIM4, TIM_EncoderMode_TI12, TIM_ICPolarity_Rising, TIM_ICPolarity_Rising);//使用编码器模式3
  TIM_ICStructInit(&TIM_ICInitStructure);
  TIM_ICInitStructure.TIM_ICFilter = 10;
  TIM_ICInit(TIM4, &TIM_ICInitStructure);
  TIM_ClearFlag(TIM4, TIM_FLAG_Update);//清除TIM的更新标志位
  TIM_ITConfig(TIM4, TIM_IT_Update, ENABLE);
  //Reset counter
  TIM_SetCounter(TIM4,0);
  TIM_Cmd(TIM4, ENABLE);
}
```

图 5-2-3 编码器 TIM4 的初始化

因为D157B使用的是电机接线来连接电机，所以我们的例程也包括了编码器，实现一个闭环的操作。并且在主函数处串口打印出我们的编码器值和电源电压的值。注意：编码器模式下只能对应定时器的 CH1 和 CH2 通道，刚好对应的是 A、B 相。

```c
/*****************************************************************************
函数功能：电机的闭环控制
入口参数：左右电机的编码器值
返回值   ：电机的PWM
*****************************************************************************/
int Velocity_A(int TargetVelocity, int CurrentVelocity)
{
    int Bias;  //定义相关变量
        static int ControlVelocity, Last_bias; //静态变量，函数调用结束后其值依然存在

        Bias=TargetVelocity-CurrentVelocity; //求速度偏差

        ControlVelocity+=Velcity_Kp*(Bias-Last_bias)+Velcity_Ki*Bias;  //增量式PI控制器
                                                                        //Velcity_Kp*(Bias-Last_bias) 作用为限制加速度
                                                                          //Velcity_Ki*Bias  速度控制值由Bias不断积分得到 偏差越大加速度越大

        Last_bias=Bias;
        if(ControlVelocity>7200) ControlVelocity=7200;
        else if(ControlVelocity<-7200) ControlVelocity=-7200;
        return ControlVelocity; //返回速度控制值
}
```

```c
int TargetVelocity=500;
float Velcity_Kp=1,  Velcity_Ki=3,  Velcity_Kd; //相关速度PID参数
```

修改此值可以修改速度

图 5-2-4 电机的闭环控制

如图 5-2-4 所示，这是一个电机的闭环控制代码，利用的是 PID 控制电机的 PWM 保持在我们想要的一个值。PID 就是比例、微分和积分控制。先计算出编码器当下的值，利用电机当下的值来和我们设定的值来进行 PID 控制，从而实现电机一直保持在我们想要的速度。这里只是对编码器值的一个反馈，主要是提出电机闭环控制的思想，并没有计算出速度与编码器的关系，所以无法进行速度的控制。

用户可以自行 DIY 出自己想要的效果，例如想要实现速度闭环的控制，就先

弄清楚速度与编码器的关系：速度=编码器读数（利用中断进行一个周期读数）*读取频率*/倍频数/减速比/编码器精度*轮子的周长。然后就可以根据输入的速度来反向计算出编码器的值（运动学逆运算）。

```
void Set_PWM(int motor_left,int motor_right)
{
    if(motor_left>0)
    {
        PWMB_IN1=7200;
        PWMB_IN2=7200-motor_left;
    }
    else
    {
        PWMB_IN1=7200+motor_left;
        PWMB_IN2=7200;
    }
    if(motor_right>0)
    {
        PWMA_IN1=7200;
        PWMA_IN2=7200-motor_right;
    }
    else
    {
        PWMA_IN1=7200+motor_right;
        PWMA_IN2=7200;
    }


}
```

图 5-2-5 电机的控制

这就是对应了表 3-1-3 中的慢衰竭模式，一个为高电平，一个为 PWM 输入。